

Applied Machine Learning

Comprehensive Exam Revision Notes

Based on Lecture Slides (Weeks 1–10) & Practice Exam

Auto-generated from course materials (Brooks Paige, Matt Kusner)

Contents

1	Introduction & ML Pipeline	2
1.1	The Applied ML Workflow	2
1.2	Data Types & Preprocessing	2
1.3	Evaluation Metrics	2
1.4	Cross-Validation	2
2	Linear Models & Regularisation	3
2.1	The General Framework	3
2.2	Ordinary Least Squares	3
2.3	Ridge Regression (L_2 Regularisation)	3
2.4	LASSO (L_1 Regularisation)	4
2.5	Classification Losses	4
2.6	Multi-Class Classification	4
3	Kernel Methods	5
3.1	The Kernel Trick	5
3.2	Common Kernels	5
3.3	Kernel Ridge Regression	5
3.4	Kernel Validity	5
3.5	Random Fourier Features	6
4	Optimisation	6
4.1	Gradient Descent	6
4.2	Stochastic & Mini-Batch GD	6
4.3	Momentum	6
4.4	Adagrad	7
4.5	Newton's Method (Second Order)	7
5	Bias–Variance Decomposition	7
6	Decision Trees (CART)	8
7	Bagging & Random Forests	8
8	Boosting (Gradient Boosting & AdaBoost)	9
8.1	Gradient Boosting	9
8.2	AdaBoost	9

9	Unsupervised Learning & Clustering	10
9.1	K-Means	10
9.2	Agglomerative Clustering	10
9.3	Gaussian Mixture Models (GMMs)	10
10	PCA & Dimensionality Reduction	11
11	Matrix Factorisation	11
11.1	Non-Negative Matrix Factorisation (NNMF)	11
11.2	Autoencoders	11
12	Data Visualisation (SNE, t-SNE)	12
13	Interpretability & Explanations	12
13.1	Inherently Interpretable Models	12
13.2	Post-Hoc Explanations	12
14	Fairness in ML	13
14.1	Sources of Bias	13
14.2	Fairness Definitions	13
14.3	Counterfactual Fairness (Kusner et al. 2017)	13
15	Differential Privacy	13
16	Practice Exam: Key Questions & Answers	14
17	Cross-Topic Connections	15
17.1	Common Pitfalls Summary	15

Introduction & ML Pipeline

The Applied ML Workflow

The Full Pipeline (all steps matter)

1. Learn about the domain / application
 2. Identify an appropriate task
 3. Collect data
 4. Clean and preprocess data
 5. Transform data or select useful features
 6. Choose an appropriate method
 7. **Train the model** (“fancy machine learning”)
 8. Evaluate, visualise, and interpret results
 9. Deploy and monitor performance
- Typically you go through repeated cycles of these steps.

Central goal: minimise *test* error, not training error. Memorising the training set = overfitting.
Fundamental assumption: test data is similar to training data.

Data Types & Preprocessing

Numerical preprocessing: scaling/centering (whitening), rank transformation, clipping, nonlinear transforms ($\log$, $\sqrt{\cdot}$). Be careful scaling sparse data. Fixed transforms: $\log(x) : \mathbb{R}^+ \rightarrow \mathbb{R}$, $\text{logit}(x) : [0, 1] \rightarrow \mathbb{R}$ where $\text{logit}(x) = \log \frac{x}{1-x}$.

Categorical preprocessing: one-hot encoding, label/ordinal encoding (only for truly ordered categories), frequency encoding, target/mean encoding (pitfall: leakage), learned embeddings (end-to-end with neural nets).

Evaluation Metrics

Classification Metrics

Accuracy: $\frac{1}{N} \sum_{i=1}^N \mathbb{I}[f(x_i) = y_i]$

- Hard predictions; can't be optimised with gradients; misleading for imbalanced classes.

Precision = $\frac{TP}{TP+FP}$, **Recall** = $\frac{TP}{TP+FN}$, **Specificity** = $\frac{TN}{TN+FP}$.

AUC-ROC: area under TPR vs. FPR curve across all thresholds.

Log-loss (binary cross-entropy):

$$\ell(\theta) = - \sum_{i=1}^N [y_i \log f_{\theta}(x_i) + (1 - y_i) \log(1 - f_{\theta}(x_i))].$$

Regression Metrics

MSE: $\frac{1}{N} \sum (f(x_i) - y_i)^2$. Assumes Gaussian errors; best constant predictor = mean.

MAE: $\frac{1}{N} \sum |f(x_i) - y_i|$. Assumes Laplace errors; best constant predictor = median; more robust to outliers.

Cross-Validation

K-fold CV: split into *K* folds, train on *K*−1, test on 1, rotate. For non-iid data: stratified sampling (grouped data), temporal splits (time series — always test on future data).

△ Don't look at the test set

Even computing scaling statistics on test data leaks information. Fit all transforms on the training set only.

Exam Q&A:

Intro **Q1**: Why is accuracy poor for imbalanced datasets?

A: If 99% of examples are class 0, always predicting 0 gives 99% accuracy but is useless. Precision, recall, AUC-ROC better capture performance.

Q2: Why use temporal splits for time-series data?

A: Random splits test interpolation; deployment requires extrapolation. Temporal splits simulate this by keeping validation/test data strictly in the future.

Q3: What is the fundamental assumption for ML generalisation?

A: Test data comes from the same (or similar) distribution as training data.

Linear Models & Regularisation

The General Framework

Regularised Empirical Risk Minimisation

$$\min_{\mathbf{w}} L(\mathbf{w}) = \sum_{i=1}^N \ell(\mathbf{w}^\top \phi(x_i), y_i) + \lambda r(\mathbf{w}).$$

Mix and match different losses ℓ and regularisers r .

Ordinary Least Squares

OLS Derivation

Loss: $L(\mathbf{w}) = \|\mathbf{y} - \Phi\mathbf{w}\|_2^2$. Derivative:

$$\frac{\partial L}{\partial \mathbf{w}} = 2\Phi^\top \Phi \mathbf{w} - 2\Phi^\top \mathbf{y}.$$

Set to zero: $\mathbf{w} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{y}$ (the normal equations).

△ OLS fails when $D > N$

$\Phi^\top \Phi \in \mathbb{R}^{D \times D}$ has rank $\leq N < D$, so it is singular and non-invertible. The system is underdetermined with infinitely many solutions. Ridge regression fixes this.

Ridge Regression (L_2 Regularisation)

Ridge Regression

$$L(\mathbf{w}) = \sum_{i=1}^N (y_i - \mathbf{w}^\top \phi(x_i))^2 + \lambda \mathbf{w}^\top \mathbf{w}.$$

Closed-form: $\mathbf{w} = (\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top \mathbf{y}$.

- Always invertible, even when $D > N$.
- Shrinks weights toward zero but never exactly to zero.
- With redundant (identical) features: distributes weight equally, $w_1 = w_2$.

LASSO (L_1 Regularisation)

LASSO

$$L(\mathbf{w}) = \sum_{i=1}^N (y_i - \mathbf{w}^\top \phi(x_i))^2 + \lambda \|\mathbf{w}\|_1.$$

- Non-differentiable at zero $\Rightarrow$ needs specialised optimisers (coordinate descent).
- Produces **sparse** solutions: some weights exactly zero $\Rightarrow$ feature selection.
- With redundant features: arbitrarily selects one and zeros the other.

$\triangle$ LASSO sparsity $\neq$ feature importance

$w_d = 0$ does **not** mean “feature d is unimportant” — it could be correlated with a selected feature.

Classification Losses

For binary labels $y_i \in \{+1, -1\}$:

Binary Classification Losses

Logistic loss: $\ell = \log(1 + \exp(-\mathbf{w}^\top \phi(x_i) \cdot y_i))$ — smooth, differentiable.

Hinge loss: $\ell = \max(1 - \mathbf{w}^\top \phi(x_i) \cdot y_i, 0)$ — used in SVM.

SVM objective (hinge + L_2):

$$\min_{\mathbf{w}} \sum_{i=1}^N \max(1 - y_i \mathbf{w}^\top \phi(x_i), 0) + \lambda \|\mathbf{w}\|_2^2.$$

Maximises the margin γ ; points at the margin are *support vectors*.

Multi-Class Classification

Logits $\mathbf{z}_i = W^\top \phi(x_i) \in \mathbb{R}^K$, softmax $\hat{y}_{i,k} = \frac{\exp(z_{i,k})}{\sum_j \exp(z_{i,j})}$, cross-entropy loss $\ell = -\sum_k y_{i,k} \log \hat{y}_{i,k}$.

Exam Q&A:

Linear Models **Q1 (Derivation)**: Derive the ridge regression closed-form.

A: Expand $L = \mathbf{w}^\top (\Phi^\top \Phi + \lambda I) \mathbf{w} - 2 \mathbf{w}^\top \Phi^\top \mathbf{y} + \text{const}$. Differentiate: $2(\Phi^\top \Phi + \lambda I) \mathbf{w} - 2 \Phi^\top \mathbf{y} = 0$. Solve: $\mathbf{w} = (\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top \mathbf{y}$.

Q2 (Comparison): Compare L_1 and L_2 regularisation.

A: L_2 shrinks all weights smoothly, has a closed form, splits weight among correlated features. L_1 drives some weights to exactly zero (sparsity/feature selection) but requires iterative optimisation and arbitrarily selects among correlated features.

Q3: What happens to OLS when $D > N$?

A: $\Phi^\top \Phi$ is singular ($\text{rank} \leq N < D$), so the system is underdetermined with infinitely many

solutions. Ridge adds λI to make it full-rank.

Kernel Methods

The Kernel Trick

Core Idea

The optimal weights can be expressed as $\mathbf{w} = \sum_{i=1}^N \alpha_i \phi(x_i)$, so all predictions depend only on inner products $\phi(x_i)^\top \phi(x_j)$.

Define the **kernel function** $\kappa(x_i, x_j) = \phi(x_i)^\top \phi(x_j)$ and the **kernel matrix** $K \in \mathbb{R}^{N \times N}$ with $K_{ij} = \kappa(x_i, x_j) = \Phi \Phi^\top$.

Proof (by induction on GD iterates): initialise $\mathbf{w}_0 = \mathbf{0} = \sum \alpha_i^0 \phi(x_i)$. Each update $\mathbf{w}_{t+1} = \mathbf{w}_t - \epsilon \sum \gamma_i^t \phi(x_i)$ preserves the form $\mathbf{w}_t = \sum \alpha_i^t \phi(x_i)$ with $\alpha_i^t = \alpha_i^{t-1} - \epsilon \gamma_i^{t-1}$.

Common Kernels

Kernel Functions

- **Linear:** $\kappa(x, z) = x^\top z$
- **Polynomial:** $\kappa(x, z) = (1 + x^\top z)^d$ — all monomials up to degree d
- **RBF (Gaussian):** $\kappa(x, z) = \exp(-\frac{\|x-z\|^2}{2\sigma^2})$ — infinite-dim. feature space

Kernel Ridge Regression

Kernel Ridge Regression

$$\boldsymbol{\alpha} = (K + \lambda I)^{-1} \mathbf{y}.$$

Prediction at x_* : $\hat{y} = \mathbf{y}^\top (K + \lambda I)^{-1} \mathbf{k}_*$ where $(\mathbf{k}_*)_i = \kappa(x_i, x_*)$.

Cost: $O(N^3)$ for inversion — compare $O(ND^2)$ for primal.

Kernel Validity

Validity Conditions (equivalent)

K is a valid kernel matrix iff it is **positive semi-definite (PSD)**:

1. $\exists B$ such that $K = BB^\top$
2. $\forall \mathbf{q}, \mathbf{q}^\top K \mathbf{q} \geq 0$
3. All eigenvalues of K are ≥ 0

△ Symmetric $\neq$ valid kernel

A real symmetric matrix can have negative eigenvalues, which would make it *not* a valid kernel.

Composition rules: given valid κ_1, κ_2 : $c\kappa_1$ ($c > 0$), $\kappa_1 + \kappa_2$, $\kappa_1 \cdot \kappa_2$, $f(x)\kappa_1 f(z)$, $\exp(\kappa_1)$, $g(\kappa_1)$ for polynomial g with positive coefficients, etc.

Random Fourier Features

Approximating the RBF Kernel

Set $\phi(x) = \cos(Ax + b)$ with $A_{ij} \sim \mathcal{N}(0, 1)$, $b_i \sim \text{Uniform}[0, 2\pi]$. Then:

$$\mathbb{E}[\phi(x)^\top \phi(z)] = \kappa(x, z).$$

Creates an explicit D -dimensional feature space. If $D > N$ we'd be better off with exact kernels.

Exam Q&A:

Kernels Q1: What is necessary and sufficient for K to be a valid kernel?

A: K must be PSD: all eigenvalues ≥ 0 , equivalently $\mathbf{q}^\top K \mathbf{q} \geq 0$ for all $\mathbf{q}$.

Q2: When use kernel SVM with linear kernel vs primal SVM?

A: When $N \ll D$. The kernel form works in $N \times N$ space ($O(N^3)$) while primal works in D -dim space. If D is very large but N small, the kernel formulation is cheaper.

Q3: As $d \rightarrow \infty$, what happens to the support vector ratio m/n for an RBF-kernel SVM?

A: With the RBF kernel (already infinite-dim), as input d increases, data becomes more spread out and the kernel matrix approaches the identity. $m/n \rightarrow 1$ (all points become support vectors).

Optimisation

Gradient Descent

Gradient Descent

$$\theta_{t+1} = \theta_t - \epsilon \nabla L(\theta_t).$$

From first-order Taylor: $L(\theta + \delta) \approx L(\theta) + \delta^\top \nabla L(\theta)$. Choosing $\delta = -\epsilon \nabla L$ gives $L(\theta_{t+1}) \approx L(\theta_t) - \epsilon \|\nabla L\|^2 \leq L(\theta_t)$.

Stochastic & Mini-Batch GD

When $L = \frac{1}{N} \sum L_i$, use a single or mini-batch of M examples: $\theta_{t+1} = \theta_t - \epsilon \nabla L_j(\theta)$.

Large batch $\rightarrow$ less noise, fewer iterations. Small batch $\rightarrow$ faster updates.

△ SGD loss can increase on a single step

Gradient estimates are noisy. A single increase does **not** indicate divergence — monitor a moving average.

Momentum

Momentum

$$\tilde{g}_{t+1} = \mu \tilde{g}_t - \epsilon \nabla L(\theta_t), \quad \theta_{t+1} = \theta_t + \tilde{g}_{t+1}.$$

Averages recent gradients $\Rightarrow$ reduces oscillation, crosses saddle points, accelerates in flat regions.

Adagrad

Adagrad

$$G_{t,ii} = \sum_{\tau=1}^t g_{\tau,i}^2, \quad \theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii} + \varepsilon}} g_{t,i}.$$

Per-parameter adaptive learning rate. Problem: G accumulates forever, learning rate $\rightarrow 0$. Fixes: RMSprop (exponential moving average), Adam (momentum + RMSprop).

Newton's Method (Second Order)

Newton's Method

$$\theta_{t+1} = \theta_t - \epsilon H^{-1} \nabla L(\theta_t),$$

where $H = \nabla^2 L$ is the Hessian. Converges faster but $O(D^2)$ memory, $O(D^3)$ per step. Not guaranteed downhill unless $H \succ 0$. Not useful for stochastic objectives.

Gauss-Newton: approximate $H \approx 2 \sum \nabla f_n \nabla f_n^\top$ (outer product). Cheaper, guaranteed PSD.

Quasi-Newton (L-BFGS): iteratively approximate H^{-1} without computing the full Hessian.

Exam Q&A:

Optimisation Q1: Why might the loss increase on one SGD iteration?

A: Stochastic gradients have nonzero variance. Individual updates may not point in the true gradient direction. This is normal; monitor a moving average.

Q2: Compare GD, SGD, Newton.

A: GD uses full gradient ($O(N)$ /step, deterministic). SGD uses noisy estimate ($O(M)$ /step, faster wall-clock). Newton uses the Hessian ($O(D^3)$ /step, much faster convergence for convex functions, impractical for large D or stochastic objectives).

Bias-Variance Decomposition

The Decomposition (squared-error loss)

Let $h_{\mathcal{D}} = A(\mathcal{D})$, $\bar{h}(x) = \mathbb{E}_{\mathcal{D}}[h_{\mathcal{D}}(x)]$, $\bar{y}(x) = \mathbb{E}_{y|x}[Y]$. Then:

$$\underbrace{\mathbb{E}_{x,y,\mathcal{D}}[(h_{\mathcal{D}}(x) - y)^2]}_{\text{expected test error}} = \underbrace{\mathbb{E}_{x,\mathcal{D}}[(h_{\mathcal{D}}(x) - \bar{h}(x))^2]}_{\text{variance}} + \underbrace{\mathbb{E}_x[(\bar{h}(x) - \bar{y}(x))^2]}_{\text{bias}^2} + \underbrace{\mathbb{E}_y[(\bar{y}(x) - y)^2]}_{\text{noise}}.$$

Derivation: add/subtract $\bar{h}$: $(h_{\mathcal{D}} - y)^2 = ((h_{\mathcal{D}} - \bar{h}) + (\bar{h} - y))^2$. Cross-term vanishes since $\mathbb{E}_{\mathcal{D}}[h_{\mathcal{D}} - \bar{h}] = 0$. Repeat for $(\bar{h} - y)^2$ by adding/subtracting $\bar{y}$.

Detecting the regime:

- **High variance:** training error $\ll$ test error (overfitting). Fix: more data, bagging, reduce complexity.
- **High bias:** training error $\approx$ test error, both high (underfitting). Fix: more features, boosting, increase complexity.

Exam Q&A:

Bias-Variance Q1: Prove the cross-term $\mathbb{E}[(h_{\mathcal{D}} - \bar{h})(\bar{h} - y)] = 0$.

A: Factor: $\mathbb{E}_{x,y}[\mathbb{E}_{\mathcal{D}}[h_{\mathcal{D}} - \bar{h}] \cdot (\bar{h} - y)]$. Inner expectation: $\mathbb{E}_{\mathcal{D}}[h_{\mathcal{D}}] - \bar{h} = \bar{h} - \bar{h} = 0$.

Q2: A deep tree has near-zero training error but high test error. Diagnosis?

A: High variance (overfitting). Fix: limit depth, prune, bag/random forest, more data.

Decision Trees (CART)

CART Algorithm

For a node containing set S , for all candidate splits $t = (f, c)$:

$$S_L^t = \{(x, y) \in S : x_f \leq c\}, \quad S_R^t = \{(x, y) \in S : x_f > c\}.$$

Regression impurity (squared loss): $L(S) = \frac{1}{|S|} \sum_{(x,y) \in S} (y - \bar{y}_S)^2$.

Pick split minimising $L(S_L) + L(S_R)$. Recurse into children until stopping criterion.

△ Trees are myopic

Greedy splitting cannot “look ahead.” A split that appears useless now may enable a good subsequent split (e.g. XOR requires a non-improving first split). Do **not** stop splitting just because the current split doesn't improve impurity.

Finding the minimum-size tree is NP-hard.

Exam Q&A:

CART Q1: Is a depth-1 tree a linear model?

A: No. A stump thresholds a single feature (step function, axis-aligned). A linear model uses a hyperplane over all features simultaneously.

Q2: Prove $t = \frac{1}{m} \sum y_i$ minimises the squared loss at a leaf.

A: $\partial L / \partial t = \sum 2(t - y_i) = 2(mt - \sum y_i) = 0 \Rightarrow t = \bar{y}$. $\partial^2 L / \partial t^2 = 2m > 0$, confirming minimum.

Q3: When does a CART tree (unlimited depth) achieve 0% training error?

A: When no two training points share identical feature vectors but different labels.

Bagging & Random Forests

Bagging Algorithm

1. Sample m datasets $\mathcal{D}_1, \dots, \mathcal{D}_m$ with replacement from $\mathcal{D}$ (each $|\mathcal{D}_i| = N$).
 2. Train model h_i on each $\mathcal{D}_i$.
 3. Predict: $\hat{h}(x) = \frac{1}{m} \sum_{i=1}^m h_i(x)$.
- Motivated by LLN: $\frac{1}{m} \sum h_i \rightarrow \bar{h}$ as $m \rightarrow \infty \Rightarrow$ **reduces variance**.

Out-of-bag error: each point is excluded from $\sim 37\%$ of bootstrap samples. Average predictions from those models to estimate test error *without* a held-out set:

$$\varepsilon_{\text{OOB}} = \frac{1}{N} \sum_{i=1}^N (\tilde{h}(x_i) - y_i)^2, \quad \tilde{h}(x_i) = \frac{1}{|S_i|} \sum_{k \in S_i} h_k(x_i), \quad S_i = \{k \mid (x_i, y_i) \notin \mathcal{D}_k\}.$$

Random Forests

Bagged full-depth trees + random feature subsampling at each split:

- At each split, randomly sample $k \leq D$ features and only consider those.
- Rule of thumb: $k = \sqrt{D}$.
- Only 2 hyperparameters (m, k), very insensitive to settings.
- One of the best “off-the-shelf” classifiers.

Exam Q&A:

Bagging & RF **Q:** Why does random feature subsampling help beyond standard bagging?

A: It decorrelates the trees. Standard bagging can produce correlated trees if one feature dominates (every tree splits on it first). Random subsampling forces different features, increasing the effective variance reduction from averaging.

Boosting (Gradient Boosting & AdaBoost)

Gradient Boosting

Boosting = Gradient Descent in Function Space

Build ensemble $H_T(x) = \sum_{t=1}^T \alpha_t h_t(x)$ iteratively. At each step:

$$h_{t+1} = \arg \min_h \ell(H_t + \alpha h) \approx \arg \min_h \sum_{i=1}^N r_i h(x_i), \quad r_i = \frac{\partial \ell}{\partial [H(x_i)]}.$$

Gradient Boosted Regression Trees (GBRT)

With squared-error loss $\ell(H) = \frac{1}{2} \sum (H(x_i) - y_i)^2$:

$$t_i = -r_i = y_i - H_t(x_i) \quad (\text{the residuals}).$$

Each new tree is fit to predict the residuals: $h_{t+1} = \arg \min_h \sum (h(x_i) - t_i)^2$.

AdaBoost

AdaBoost (exponential loss, $y \in \{+1, -1\}$)

Loss: $\ell(H) = \sum_{i=1}^N e^{-y_i H(x_i)}$. Instance weights: $w_i \propto e^{-y_i H(x_i)}$.

Best weak learner: minimise weighted classification error $\varepsilon = \sum_{h(x_i) \neq y_i} w_i$.

Closed-form step size:

$$\alpha = \frac{1}{2} \log \frac{1 - \varepsilon}{\varepsilon}$$

Weight update: $w_i \leftarrow w_i \cdot \frac{e^{-\alpha y_i h(x_i)}}{\sqrt{\varepsilon(1-\varepsilon)}}$.

Training loss bound: $\ell(H_T) \leq N [2\sqrt{c(1-c)}]^T$ with $c = \max_t \varepsilon_t < \frac{1}{2} \Rightarrow$ **exponential convergence**. After $O(\log N)$ iterations, training error = 0.

Exam Alert: AdaBoost step-size derivation

Minimise $\ell(H + \alpha h) = \sum e^{-y_i[H(x_i) + \alpha h(x_i)]}$. Differentiate w.r.t. α :

$$-\sum y_i h(x_i) e^{-y_i[H(x_i) + \alpha h(x_i)]} = 0.$$

Split into correct ($y_i h(x_i) = +1$) and incorrect ($= -1$):

$$-(1 - \varepsilon)e^{-\alpha} + \varepsilon e^{\alpha} = 0 \Rightarrow e^{2\alpha} = \frac{1 - \varepsilon}{\varepsilon} \Rightarrow \alpha = \frac{1}{2} \log \frac{1 - \varepsilon}{\varepsilon}.$$

Exam Q&A:

Boosting Q: Why does AdaBoost upweight misclassified examples?

A: $w_i \propto \exp(-y_i H(x_i))$. When H misclassifies x_i ($y_i H(x_i) < 0$), the exponent is positive and the weight increases. The next weak learner focuses on these hard examples.

Unsupervised Learning & Clustering

K-Means

K-Means

Objective: $\min \text{WCC} = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$.

Algorithm: (1) Assign each point to nearest centre. (2) Recompute centres as cluster means. Repeat until convergence.

Limitations: spherical clusters only, sensitive to initialisation (run multiple times), sensitive to scaling, no principled way to choose K , centres may not be data points.

△ K-means centres $\neq$ data points

Centres are means of cluster members. Use **K-medoids** to constrain centres to actual data points.

Agglomerative Clustering

Bottom-up: greedily merge the two most similar clusters. Linkage rules:

- **Single:** $D_{\text{SL}} = \min_{x \in G, x' \in H} d(x, x')$ — chaining, elongated clusters.
- **Complete:** $D_{\text{CL}} = \max$ — compact clusters.
- **Average:** mean pairwise distance.
- **Ward's:** minimise increase in within-cluster variance — even-sized clusters, resembles K-means.

Gaussian Mixture Models (GMMs)

GMM

$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x | \mu_k, \Sigma_k)$. Latent variable: $z_n \sim \text{Discrete}(\pi)$, $x_n | z_n=k \sim \mathcal{N}(\mu_k, \Sigma_k)$.

EM algorithm: E-step: soft-assign $p(z_n=k | x_n) \propto \pi_k \mathcal{N}(x_n | \mu_k, \Sigma_k)$. M-step: update μ_k, Σ_k, π_k with weighted statistics.

K-means is a special case with $\Sigma_k = \sigma^2 I$ ($\sigma \rightarrow 0$) and hard assignments.

Exam Q&A:

Clustering **Q**: After convergence, are K-means centres always original data points?

A: False. Centres are means, not actual data points.

Q: How can K-means be used as a feature extractor?

A: After fitting, create features: cluster membership (categorical) or distances to each of the K centres (K real-valued features).

PCA & Dimensionality Reduction

PCA

Centre data: $\tilde{X} = X - \mu$. Covariance: $S = \frac{1}{N} \tilde{X}^\top \tilde{X} = V \Lambda V^\top$.

SVD: $\tilde{X} = U D V^\top$. Columns of V are principal components.

Projection & reconstruction: $z_i = V_K^\top (x_i - \mu)$, $\hat{x}_i = V_K z_i + \mu$.

Variance explained: $\sum_{k=1}^K \lambda_k / \sum_{k=1}^D \lambda_k$.

1st PC: $\max_{\|u\|=1} u^\top S u \Rightarrow S u = \lambda u \Rightarrow$ eigenvector with largest eigenvalue.

Properties: orthogonal projections ordered by variance; defined up to sign/rotation; sensitive to scaling; linear only.

PCA as generative model: $z \sim \mathcal{N}(0, I_K)$, $x | z \sim \mathcal{N}(Wz + \mu, \sigma^2 I)$. Marginal: $p(x) = \mathcal{N}(x | \mu, WW^\top + \sigma^2 I)$.

Exam Q&A:

PCA **Q (T/F)**: PCA is always better than random projection.

A: False. PCA maximises variance, but high-variance directions may not be discriminative. Random projections preserve pairwise distances (JL lemma).

Q: Why can PCA detect outliers?

A: Project and reconstruct. Points with high reconstruction error don't fit the principal subspace — likely outliers.

Matrix Factorisation

Non-Negative Matrix Factorisation (NNMF)

NNMF

$X \approx WH$, with $W \geq 0$, $H \geq 0$ element-wise.

Poisson log-likelihood loss: $L(W, H) = \sum_{i,j} [x_{ij} \log(WH)_{ij} - (WH)_{ij}]$.

Multiplicative updates:

$$w_{ik} \leftarrow w_{ik} \cdot \frac{\sum_j h_{kj} x_{ij} / (WH)_{ij}}{\sum_j h_{kj}}, \quad h_{kj} \leftarrow h_{kj} \cdot \frac{\sum_i w_{ik} x_{ij} / (WH)_{ij}}{\sum_i w_{ik}}.$$

Non-negativity $\Rightarrow$ additive parts (face parts, musical notes, topics) rather than cancelling modes.

Autoencoders

$z_i = f_\phi(x_i)$ (encoder), $\hat{x}_i = g_\theta(z_i)$ (decoder). Loss: $L = \frac{1}{N} \sum \|x_i - g_\theta(f_\phi(x_i))\|^2$. Nonlinear extension of PCA.

Data Visualisation (SNE, t-SNE)

t-SNE

High-D transition: $p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)}$.

Symmetric: $p_{ij} = \frac{p_{j|i} + p_{i|j}}{2N}$.

Low-D (Student- t , 1 d.f.): $q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}}$.

Loss: $L = D_{\text{KL}}(p||q) = \sum_{i,j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$.

Gradient: $\frac{\partial L}{\partial y_i} = 4 \sum_j (p_{ij} - q_{ij})(1 + \|y_i - y_j\|^2)^{-1}(y_i - y_j)$.

Heavy tails of Student- t alleviate the “crowding problem”: moderately distant high-D points can spread in low-D.

△ t-SNE limitations

- No parametric mapping — new points cannot be embedded.
- Global distances between clusters are **NOT** meaningful.
- $O(N^2)$ per iteration (Barnes-Hut: $O(N \log N)$).

Interpretability & Explanations

Inherently Interpretable Models

Linear models (esp. sparse/ L_1), decision trees, K-NN, Naïve Bayes, GAMs: $\hat{y} = \beta_0 + f_1(x_1) + f_2(x_2) + \dots$.

Post-Hoc Explanations

Post-Hoc Methods

LIME (local surrogate): perturb x_* , query black-box f , fit interpretable model locally. Sensitive to perturbation definition.

Partial Dependence: $\hat{f}_d(x_d) = \frac{1}{N} \sum_{i=1}^N f(x_d, x_{\setminus d}^{(i)})$.

Permutation Feature Importance: shuffle feature d , measure loss increase $F_d = L_d/L$.

Counterfactual Explanations: find x' closest to x with $f(x') \neq f(x)$. Faithful by construction.

Saliency Maps: $|\partial f / \partial x|$ for images. Popular but unreliable (Adebayo et al. 2018).

Rudin (2019): “Stop explaining black-box models for high-stakes decisions.” In many domains, interpretable models perform comparably when proper feature engineering is done.

Exam Q&A:

Interpretability Q: Why might permutation feature importance be misleading with correlated features?

A: Shuffling feature d breaks its correlation with other features, creating unrealistic data. The model may still predict well using correlated features, making d appear unimportant.

Fairness in ML

Sources of Bias

(1) Model misuse (dataset shift). (2) Biased training data (historical discrimination). (3) Proxy features (postcode $\rightarrow$ race).

Fairness Definitions

Statistical Fairness Notions

Fairness Through Unawareness: Don't use A . *Insufficient* — proxies leak.

Demographic Parity: $P(\hat{Y}=1 \mid A=a) = P(\hat{Y}=1 \mid A=a')$.

Equality of Opportunity (Hardt et al. 2016): $P(\hat{Y}=1 \mid A=a, Y=1) = P(\hat{Y}=1 \mid A=a', Y=1)$.

Individual Fairness (Dwork et al. 2012): $D(\hat{Y}(x_i, a_i), \hat{Y}(x_j, a_j)) \leq d(x_i, x_j)$.

Key impossibility: no observational fairness notion can fully prevent discrimination — same correlations can arise from different causal structures.

Counterfactual Fairness (Kusner et al. 2017)

Counterfactual Fairness

A predictor is counterfactually fair if:

$$\hat{Y}(G, L) = \hat{Y}(G_{A \leftarrow a'}, L_{A \leftarrow a'}).$$

Three-step procedure: (1) Infer unobserved U from causal model. (2) Change $A \leftarrow a'$. (3) Recompute observed variables with new A , same U .

Practical rule: predictions using only *non-descendants* of A are counterfactually fair.

Exam Q&A:

Fairness Q: Why is Fairness Through Unawareness insufficient?

A: Other features (postcode, income) can be proxies causally influenced by the sensitive attribute. The model still learns discriminatory patterns.

Q: Why are causal models important for fairness?

A: The same correlation (race $\leftrightarrow$ loan) can arise from different causal structures requiring different interventions. Only causal reasoning can identify unfair pathways.

Differential Privacy

ϵ -Differential Privacy (Dwork et al. 2006)

A randomised algorithm $\mathcal{A}$ is ϵ -differentially private if for all $x \in \text{Range}(\mathcal{A})$ and all neighbouring D, D' (differing in one row):

$$\frac{\Pr[\mathcal{A}(D) = x]}{\Pr[\mathcal{A}(D') = x]} \leq e^\epsilon.$$

Relaxation: $\Pr[\mathcal{A}(D) = x] \leq e^\epsilon \Pr[\mathcal{A}(D') = x] + \delta$.

Global sensitivity: $\Delta f = \max_{D, D'} \|f(D) - f(D')\|$.

Privacy Mechanisms

Laplace: release $f(D) + \text{Lap}(0, \Delta f/\varepsilon)$. Guarantees $(\varepsilon, 0)$ -DP.

Gaussian: release $f(D) + \mathcal{N}(0, \sigma^2)$ with σ calibrated to $\Delta f, \varepsilon, \delta$. Guarantees (ε, δ) -DP.

Exponential (discrete outputs): $\Pr[x] \propto \exp(\frac{\varepsilon q(D, x)}{2\Delta_q})$. Guarantees ε -DP.

Randomised Response: with prob. $3/4$ report truth, with $1/4$ lie. This is $(\log 3)$ -DP since $\frac{3/4}{1/4} = 3 = e^{\log 3}$.

Key properties: (1) Composition: k runs of (ε, δ) -DP $\Rightarrow (k\varepsilon, k\delta)$ -DP. (2) Post-processing cannot degrade privacy. (3) Trade-off: smaller $\varepsilon \Rightarrow$ more noise $\Rightarrow$ less accuracy.

Exam Q&A:

Privacy **Q:** Prove randomised response is $(\log 3)$ -DP.

A: Worst case: true answer flips. If true = yes, $\Pr[\text{"yes"}] = 3/4$ (truth) vs $\Pr[\text{"yes"}] = 1/4$ (random). Ratio: $3/4 \div 1/4 = 3 = e^{\log 3} \leq e^\varepsilon$ with $\varepsilon = \log 3$.

Practice Exam: Key Questions & Answers

Q1: True/False

- (a) K-means centres are always original data points. **FALSE** — centres are means.
- (b) PCA always better than random projection. **FALSE** — high-variance $\neq$ discriminative.
- (c) 0% train error $\Rightarrow$ cannot have 100% test error. **FALSE** — memorisation + distribution shift.
- (d) DP via symmetric noise addition. **TRUE** — Laplace/Gaussian mechanisms.
- (e) k -NN error $\leq 2 \times$ Bayes optimal as $n \rightarrow \infty$. **TRUE** — Cover & Hart (1967).
- (f) Any real symmetric matrix is a valid kernel. **FALSE** — must be PSD.

Q3: CART

- (a) Depth-1 tree $\neq$ linear model (step function on one feature vs. hyperplane).
- (b) Reduce CART variance: (1) Bagging/RF, (2) Limit depth, (3) Pruning.
- (c) Rank preprocessing: decision trees are invariant to monotone transforms of individual features, so rank transformation does not change the tree structure.
- (d) 0% training error when no two points share identical x but different y .
- (e) “Myopic”: greedily selects best current split without considering future splits.
- (f) Optimal leaf prediction $t = \bar{y}$: differentiate $\sum (y_i - t)^2$, set to zero.

Q4: Kernel Methods

- (a) Kernel validity: K must be PSD (all eigenvalues ≥ 0).
- (c) Hard-margin SVM with polynomial kernel: nonlinear decision boundary separating classes (draw a curve around the class boundary).
- (d) m/n as $d \rightarrow \infty$ with RBF: $\rightarrow 1$ (all points become support vectors).
- (e) Use kernel SVM with linear kernel when $N \ll D$.
- (f) GP vs. kernel ridge (RBF): GP provides uncertainty & principled hyperparameter selection via marginal likelihood; typically better test error.

Cross-Topic Connections

Model	Loss ℓ	Regulariser r
Ridge Regression	Squared error	$L_2 (\ \mathbf{w}\ ^2)$
LASSO	Squared error	$L_1 (\ \mathbf{w}\ _1)$
Logistic Regression	Log-loss	L_2
SVM	Hinge	L_2
Ensemble Method	Targets	Base Learner
Bagging / RF	Variance	High-variance (deep trees)
Boosting	Bias	High-bias (stumps)
Unsupervised	Key Idea	Constraint
PCA	Max variance / min recon. error	Linear, orthogonal
NNMF	Additive parts	Non-negative
K-means	Min within-cluster variance	Hard assignment
GMM	Max likelihood of mixture	Soft assignment
Autoencoder	Min reconstruction error	Nonlinear (neural net)

Key links: K-means is a limiting case of GMM ($\Sigma_k = \sigma^2 I$, $\sigma \rightarrow 0$). PCA and GMM both have latent-variable Gaussian formulations. Kernel ridge = GP predictive mean. Random Fourier features bridge explicit/implicit feature spaces.

Common Pitfalls Summary

△ Master Pitfall List

1. LASSO $w=0 \neq$ unimportant feature — correlated features $\rightarrow$ arbitrary selection.
2. K-means centres $\neq$ data points — they are means, not medoids.
3. Accuracy is misleading for imbalanced data — use AUC, precision/recall.
4. SGD loss can increase on a single step — normal; don't stop early.
5. Trees are myopic — don't stop splitting because current split doesn't help (XOR).
6. t-SNE global distances are meaningless — only local structure is preserved.
7. Symmetric matrix $\neq$ valid kernel — must also be PSD.
8. Fairness through unawareness $\neq$ fairness — proxy variables carry information.
9. OLS fails when $D > N$ — use ridge regression.
10. Saliency maps unreliable — may not reflect actual model reasoning.

End of Revision Notes. Total coverage: 17 lecture files + practice exam. Good luck!